

GOOGLE CLOUD PROFESSIONAL CLOUD ARCHITECT

Section 4.4 — Managing Storage and Database Solutions

GCP Associate Cloud Engineer

Study Notes Study Notes April 2026

Exam Relevance

This topic is part of **Section 4: Ensuring successful operation of a cloud solution (~20 % of the exam)**. You must know how to manage Cloud Storage objects, set lifecycle policies, execute queries across data services, estimate costs, manage backups, and review job status.

1. Managing and Securing Objects in Cloud Storage

Object Operations

```
# Upload an object
gcloud storage cp local-file.txt gs://my-bucket/path/

# Download an object
gcloud storage cp gs://my-bucket/path/file.txt ./local-file.txt

# Copy between buckets
gcloud storage cp gs://source-bucket/file.txt gs://dest-bucket/file.txt

# Move/rename an object
gcloud storage mv gs://my-bucket/old-name.txt gs://my-bucket/new-name.txt

# Delete an object
gcloud storage rm gs://my-bucket/path/file.txt

# Delete all objects with a prefix
gcloud storage rm gs://my-bucket/logs/** --recursive

# List objects in a bucket
gcloud storage ls gs://my-bucket/
gcloud storage ls gs://my-bucket/path/ --recursive

# Get object metadata
gcloud storage objects describe gs://my-bucket/file.txt

# Set object storage class
gcloud storage objects update gs://my-bucket/file.txt \
  --storage-class=NEARLINE

# Set metadata on an object
gcloud storage objects update gs://my-bucket/file.txt \
  --custom-metadata=key1=value1,key2=value2
```

Access Control

Uniform Bucket-Level Access (Recommended)

- Disables object-level ACLs
- All access controlled through **IAM only**
- Simplifies access management

```
# Enable uniform bucket-level access
gcloud storage buckets update gs://my-bucket \
  --uniform-bucket-level-access

# Grant read access to all objects in a bucket
gcloud storage buckets add-iam-policy-binding gs://my-bucket \
  --member="user:alice@example.com" \
  --role="roles/storage.objectViewer"

# Grant write access
gcloud storage buckets add-iam-policy-binding gs://my-bucket \
  --member="serviceAccount:sa@project.iam.gserviceaccount.com" \
  --role="roles/storage.objectCreator"

# View bucket IAM policy
gcloud storage buckets get-iam-policy gs://my-bucket
```

Key Storage IAM Roles

Role	Permissions
<code>roles/storage.objectViewer</code>	Read objects and their metadata
<code>roles/storage.objectCreator</code>	Create objects (no read/delete)
<code>roles/storage.objectAdmin</code>	Full control over objects
<code>roles/storage.admin</code>	Full control over buckets and objects
<code>roles/storage.hmacKeyAdmin</code>	Manage HMAC keys

Signed URLs (Temporary Access)

```
# Generate a signed URL (valid for 1 hour)
gcloud storage sign-url gs://my-bucket/file.txt \
  --duration=1h \
  --private-key-file=key.json
```

- Provides **time-limited access** to an object without requiring authentication
- Useful for sharing files with external users or systems
- Can be configured for GET (download) or PUT (upload)

Object Versioning

```
# Enable versioning
gcloud storage buckets update gs://my-bucket --versioning

# List all versions of an object
gcloud storage ls --all-versions gs://my-bucket/file.txt

# Restore a previous version (copy it as the live version)
gcloud storage cp gs://my-bucket/file.txt#GENERATION gs://my-bucket/file.txt

# Disable versioning
gcloud storage buckets update gs://my-bucket --no-versioning
```

Retention Policies and Bucket Lock

```
# Set a retention policy (objects cannot be deleted for 90 days)
gcloud storage buckets update gs://my-bucket \
  --retention-period=90d

# Lock the retention policy (IRREVERSIBLE — cannot be shortened or removed)
gcloud storage buckets update gs://my-bucket \
  --lock-retention-period

# Remove retention policy (only if not locked)
gcloud storage buckets update gs://my-bucket \
  --clear-retention-period
```

2. Object Lifecycle Management Policies

What Are Lifecycle Policies?

Automate storage management by defining rules that: - **Transition** objects to cheaper storage classes - **Delete** objects after a specified time - **Delete** non-current versions (when versioning is enabled)

Common Lifecycle Rules

```
{
  "lifecycle": {
    "rule": [
      {
        "action": {"type": "SetStorageClass", "storageClass": "NEARLINE"},
        "condition": {"age": 30, "matchesStorageClass": ["STANDARD"]}
      },
      {
        "action": {"type": "SetStorageClass", "storageClass": "COLDLINE"},
        "condition": {"age": 90, "matchesStorageClass": ["NEARLINE"]}
      },
      {
        "action": {"type": "SetStorageClass", "storageClass": "ARCHIVE"},
        "condition": {"age": 365, "matchesStorageClass": ["COLDLINE"]}
      },
      {
        "action": {"type": "Delete"},
        "condition": {"age": 730}
      },
      {
        "action": {"type": "Delete"},
        "condition": {"isLive": false, "numNewerVersions": 3}
      }
    ]
  }
}
```

```
# Apply lifecycle policy
gcloud storage buckets update gs://my-bucket \
  --lifecycle-file=lifecycle.json

# View current lifecycle policy
gcloud storage buckets describe gs://my-bucket \
  --format="json(lifecycle)"

# Remove lifecycle policy
gcloud storage buckets update gs://my-bucket \
  --clear-lifecycle
```

Lifecycle Condition Parameters

Condition	Description
<code>age</code>	Number of days since object creation
<code>createdBefore</code>	Object created before a specific date
<code>isLive</code>	true for current, false for non-current versions
<code>matchesStorageClass</code>	Only apply to objects in specified class
<code>numNewerVersions</code>	Number of newer versions (versioned buckets)
<code>daysSinceNoncurrentTime</code>	Days since object became non-current
<code>matchesPrefix</code>	Only objects with matching prefix
<code>matchesSuffix</code>	Only objects with matching suffix

3. Executing Queries Across Data Services

Cloud SQL Queries

```
# Connect and run interactive queries
gcloud sql connect my-instance --user=root --database=mydb

# Execute a single query
gcloud sql connect my-instance --user=root <<< "SELECT * FROM users LIMIT 10;"

# Import/export data
gcloud sql export csv my-instance gs://my-bucket/export.csv \
  --database=mydb --query="SELECT * FROM users"

gcloud sql export sql my-instance gs://my-bucket/dump.sql \
  --database=mydb
```

BigQuery Queries

```
# Run a query from the command line
bq query --use_legacy_sql=false \
  'SELECT name, COUNT(*) as count
  FROM `project.dataset.table`
  GROUP BY name
  ORDER BY count DESC
  LIMIT 10'

# Run a query and save results to a table
bq query --use_legacy_sql=false \
  --destination_table=project:dataset.results \
  'SELECT * FROM `project.dataset.table` WHERE date > "2024-01-01"'

# Run a dry run (estimate bytes scanned, no cost)
bq query --use_legacy_sql=false --dry_run \
  'SELECT * FROM `project.dataset.large_table`'

# Query with parameters
bq query --use_legacy_sql=false \
  --parameter='name:STRING:Alice' \
  'SELECT * FROM `project.dataset.users` WHERE name = @name'
```

Spanner Queries

```
# Execute a SQL query
gcloud spanner databases execute-sql my-database \
  --instance=my-instance \
  --sql="SELECT * FROM Users WHERE UserId = 1"

# Execute with query parameters
gcloud spanner databases execute-sql my-database \
  --instance=my-instance \
  --sql="SELECT * FROM Users WHERE Name = @name" \
  --query-params="name=Alice"
```

Firestore Queries

Firestore queries are typically done through client SDKs or the Console. Limited CLI support:

```
# Export Firestore data to Cloud Storage
gcloud firestore export gs://my-bucket/firestore-backup \
  --collection-ids=users,orders

# Import Firestore data
gcloud firestore import gs://my-bucket/firestore-backup
```

AlloyDB Queries

```
# Connect to AlloyDB (via Cloud SQL Auth Proxy or direct)
# AlloyDB uses standard PostgreSQL clients
psql -h ALLOYDB_IP -U postgres -d mydb
```

4. Estimating Costs of Data Storage Resources

Cloud Storage Cost Factors

Factor	Description
Storage	Per GB/month, varies by class (Standard > Nearline > Coldline > Archive)
Retrieval	Per GB retrieved (free for Standard, increases for colder classes)
Operations	Per operation (Class A: write, Class B: read)
Network	Egress charges (free within same region, charged for inter-region and internet)
Early deletion	Charged if deleted before min duration (30/90/365 days)

Database Cost Factors

Service	Cost Components
Cloud SQL	vCPUs, memory, storage, HA (2x), read replicas, backups, network
Spanner	Processing units/nodes, storage, network
BigQuery	Queries (per TB scanned or slot-hours), storage (active vs long-term)
Firestore	Document reads/writes/deletes, storage, network
Bigtable	Nodes, storage (SSD or HDD), network
AlloyDB	vCPUs, memory, storage, HA, read pools

Cost Optimization Tips

- Use **lifecycle policies** to move data to cheaper storage classes
 - Use **BigQuery partitioning/clustering** to reduce bytes scanned
 - Use **committed use discounts** for Cloud SQL and Spanner
 - Use **BigQuery slot reservations** for predictable query costs
 - Enable **storage auto-increase** cautiously on Cloud SQL
 - Use **Archive class** for compliance data accessed rarely
-

5. Backing Up and Restoring Database Instances

Cloud SQL Backups

```
# Create an on-demand backup
gcloud sql backups create --instance=my-instance

# List backups
gcloud sql backups list --instance=my-instance

# Describe a backup
gcloud sql backups describe BACKUP_ID --instance=my-instance

# Restore from a backup
gcloud sql backups restore BACKUP_ID \
  --restore-instance=my-instance

# Restore to a different instance
gcloud sql backups restore BACKUP_ID \
  --restore-instance=my-new-instance \
  --backup-instance=my-instance

# Delete a backup
gcloud sql backups delete BACKUP_ID --instance=my-instance
```

Automated Backups

```
# Configure automated backups
gcloud sql instances patch my-instance \
  --backup-start-time=02:00 \
  --retained-backups-count=7 \
  --enable-bin-log # Required for PITR (MySQL)
```

Point-in-Time Recovery (PITR)

```
# Restore to a specific point in time
gcloud sql instances clone my-instance my-restored-instance \
  --point-in-time="2024-06-15T10:30:00.000Z"
```

- Requires **binary logging** (MySQL) or **WAL archiving** (PostgreSQL)
- Can restore to any second within the retention window (default: 7 days)

Firestore Backups

```
# Export Firestore database to Cloud Storage
gcloud firestore export gs://my-bucket/firestore-backup

# Export specific collections
gcloud firestore export gs://my-bucket/firestore-backup \
  --collection-ids=users,orders

# Import (restore) from a backup
gcloud firestore import gs://my-bucket/firestore-backup

# Schedule regular exports using Cloud Scheduler + Cloud Functions
```

Important: Firestore export/import is the primary backup mechanism. There are no built-in automated backup schedules — you must implement them using Cloud Scheduler.

6. Reviewing Job Status

Dataflow Job Status

```
# List all Dataflow jobs
gcloud dataflow jobs list --region=us-central1

# List only running jobs
gcloud dataflow jobs list --region=us-central1 --status=active

# Describe a job (detailed info)
gcloud dataflow jobs describe JOB_ID --region=us-central1

# Show job metrics
gcloud dataflow jobs describe JOB_ID \
  --region=us-central1 \
  --format="json(currentStateTime,currentState)"

# Cancel a job
gcloud dataflow jobs cancel JOB_ID --region=us-central1

# Drain a streaming job (process remaining, then stop)
gcloud dataflow jobs drain JOB_ID --region=us-central1
```

Dataflow Job States

State	Description
JOB_STATE_RUNNING	Job is actively processing
JOB_STATE_DONE	Batch job completed successfully
JOB_STATE_FAILED	Job failed
JOB_STATE_CANCELLED	Job was cancelled
JOB_STATE_DRAINING	Streaming job is draining
JOB_STATE_DRAINED	Streaming job has finished draining
JOB_STATE_UPDATED	Job was replaced by an updated version

BigQuery Job Status

```
# List recent jobs
bq ls -j --max_results=20

# Show job details
bq show -j JOB_ID

# Show jobs with specific state
bq ls -j --filter="done" --max_results=10

# Wait for a job to complete
bq wait JOB_ID

# Cancel a running job
bq cancel JOB_ID
```

BigQuery Job Types

Type	Description
Query	SQL query execution
Load	Loading data into BigQuery
Extract	Exporting data from BigQuery
Copy	Copying tables

8. Cloud SQL High Availability and Connection Methods

HA Architecture

- HA Cloud SQL instance has a primary and a standby in a different zone
- Automatic failover within ~60 seconds (DNS CNAME switches)
- `--availability-type=REGIONAL` enables HA

```
gcloud sql instances patch MY_INSTANCE --availability-type=REGIONAL
# Manual failover for testing
gcloud sql instances failover MY_INSTANCE
```

- Read replicas (not for HA — for read scaling):

```
gcloud sql instances create my-replica --master-instance-name=MY_INSTANCE --region=us-c
```

- PITR: point-in-time recovery using binary logs; enable with `--enable-point-in-time-recovery`; restore:

```
gcloud sql instances clone MY_INSTANCE MY_CLONE --point-in-time="2024-06-01T12:00:00Z"
```

Connection Methods

```
# Cloud SQL Auth Proxy (recommended for applications — handles TLS and IAM auth)
./cloud-sql-proxy MY_PROJECT:us-central1:MY_INSTANCE &
# Application connects to 127.0.0.1:5432

# Direct with SSL
gcloud sql connect MY_INSTANCE --user=postgres --database=mydb

# Private IP: set at instance creation or patch
gcloud sql instances patch MY_INSTANCE --network=projects/PROJECT/global/networks/VPC_NAME
```

9. Bigtable Operations

```
# List instances and clusters
gcloud bigtable instances list
gcloud bigtable clusters list --instances=MY_INSTANCE
# Scale cluster (nodes)
gcloud bigtable clusters update MY_CLUSTER --instance=MY_INSTANCE --num-nodes=5
# cbt tool operations
cbt -instance=MY_INSTANCE listinstances
cbt -instance=MY_INSTANCE ls                # list tables
cbt -instance=MY_INSTANCE read MY_TABLE    # read all rows
cbt -instance=MY_INSTANCE count MY_TABLE   # count rows
cbt -instance=MY_INSTANCE deleteallrows MY_TABLE
```

- GC policy per column family: keep last N versions or keep data newer than N days

```
cbt -instance=MY_INSTANCE setgcpolicy MY_TABLE cf1 "maxversions=1"
cbt -instance=MY_INSTANCE setgcpolicy MY_TABLE cf2 "maxage=30d"
```

10. Memorystore Operations

```
# Redis
gcloud redis instances list --region=us-central1
gcloud redis instances describe MY_INSTANCE --region=us-central1
gcloud redis instances export gs://MY_BUCKET/redis.rdb MY_INSTANCE --region=us-central1
gcloud redis instances import gs://MY_BUCKET/redis.rdb MY_INSTANCE --region=us-central1
gcloud redis instances upgrade MY_INSTANCE --redis-version=redis_7_0 --region=us-central1
```

- Connection: use private IP from same VPC; retrieve with `gcloud redis instances describe ... | grep host`
- Exam tip: Memorystore Redis STANDARD_HA tier provides a read replica in a different zone; BASIC has no replica

Exam Practice Questions

1. You need to automatically transition Cloud Storage objects from Standard to Nearline after 30 days and delete them after 1 year. How?

- Answer: Create a lifecycle policy with two rules: one `SetStorageClass` to NEARLINE at age 30, and one `Delete` at age 365. Apply with `gcloud storage buckets update --lifecycle-file`.
2. **A regulatory requirement mandates that certain files cannot be deleted for 7 years. How do you enforce this?**
- Answer: Set a **retention policy** of 7 years on the bucket and **lock it**. Locked retention policies cannot be shortened or removed, ensuring compliance.
3. **You want to estimate the cost of a BigQuery query before running it. How?**
- Answer: Use `bq query --dry_run` with your query. It returns the number of bytes that would be scanned without actually running the query or incurring charges.
4. **Your Cloud SQL instance needs point-in-time recovery capability. What must be enabled?**
- Answer: Enable **automated backups** (`--backup-start-time`) and **binary logging** (`--enable-bin-log` for MySQL) or WAL archiving for PostgreSQL.
5. **A Dataflow streaming job needs to be stopped, but remaining in-flight data should be processed first. What should you do?**
- Answer: Use `gcloud dataflow jobs drain JOB_ID`. This processes remaining data in the pipeline before stopping, unlike `cancel` which stops immediately.
6. **You need regular automated backups of a Firestore database. What's the recommended approach?**
- Answer: Use **Cloud Scheduler** to trigger a **Cloud Function** that runs `gcloud firestore export` on a schedule (e.g., daily), exporting to a Cloud Storage bucket.
7. **How do you prevent users from accidentally making Cloud Storage objects public?**
- Answer: Enable **uniform bucket-level access** (removes ACLs) and use an **organization policy** (`constraints/storage.uniformBucketLevelAccess`) to enforce it across all buckets.
-

Glossary

ACL (Access Control List): A set of rules attached to a Cloud Storage object or bucket that specifies which users or groups can access it and what actions they can perform; superseded by IAM when uniform bucket-level access is enabled.

AlloyDB: A fully managed, PostgreSQL-compatible database service on Google Cloud, optimized for high-performance transactional and analytical workloads, supporting standard PostgreSQL clients.

Archive (storage class): The coldest and cheapest Cloud Storage class, designed for data accessed less than once a year, with a minimum storage duration of 365 days and high retrieval costs.

Automated backup: A scheduled backup of a Cloud SQL instance created automatically at a configured time, used to restore the instance or enable point-in-time recovery.

BigQuery: Google Cloud's fully managed, serverless data warehouse service that enables fast SQL queries over large datasets, billed by bytes scanned or slot-hours consumed.

BigQuery clustering: A table optimization technique where data is physically sorted by one or more columns, reducing the bytes scanned for queries that filter on those columns.

BigQuery partitioning: Dividing a BigQuery table into segments (by date, integer range, or ingestion time) so that queries can skip irrelevant partitions and scan fewer bytes.

BigQuery slot reservations: A commitment to a fixed number of BigQuery processing units (slots) for predictable query cost instead of paying per byte scanned.

Binary logging: A MySQL feature that records all changes to the database in a binary log file; required on Cloud SQL MySQL instances to enable point-in-time recovery.

Bigtable: Google Cloud's fully managed, wide-column NoSQL database service designed for high-throughput, low-latency workloads such as time-series, IoT, and analytics data.

Bucket Lock: A feature that permanently locks a Cloud Storage retention policy so it cannot be shortened or removed, used for compliance with data retention regulations.

cbt: The command-line tool for interacting with Cloud Bigtable, used to list tables, read rows, count rows, set GC policies, and perform other table-level operations.

CIDR (Classless Inter-Domain Routing): A notation for specifying IP address ranges (e.g., `10.0.0.0/24`); used in Cloud Storage network egress contexts and as a general networking concept.

Cloud Function: A serverless, event-driven compute service on Google Cloud used to run small pieces of code in response to events such as Pub/Sub messages or HTTP requests.

Cloud Scheduler: A fully managed, enterprise-grade cron job scheduler on Google Cloud, used to trigger Cloud Functions, Pub/Sub messages, or HTTP endpoints on a defined schedule.

Cloud SQL: Google Cloud's fully managed relational database service supporting MySQL, PostgreSQL, and SQL Server instances, with built-in backups, replication, and high availability.

Cloud SQL Auth Proxy: A client-side proxy binary that handles TLS encryption and IAM-based authentication for Cloud SQL connections, eliminating the need to manage SSL certificates or allowlisted IPs.

Coldline (storage class): A Cloud Storage class designed for data accessed at most once every 90 days, with lower storage cost than Nearline but higher retrieval cost and a 90-day minimum storage duration.

Committed use discount: A pricing discount offered for Cloud SQL and Cloud Spanner when committing to use a resource for a 1- or 3-year term.

Dataflow: Google Cloud's fully managed stream and batch data processing service, based on Apache Beam, used for ETL pipelines, real-time analytics, and data enrichment.

DNS CNAME: A type of DNS record that maps an alias name to the true (canonical) domain name; used by Cloud SQL HA to redirect connections to the standby during failover.

Early deletion fee: A charge applied when a Cloud Storage object stored in Nearline, Coldline, or Archive class is deleted before the minimum storage duration (30, 90, or 365 days respectively).

Egress: Network traffic that leaves a Google Cloud region or Google's network; subject to charges when crossing region boundaries or going to the internet.

Firestore: Google Cloud's fully managed, serverless, document-oriented NoSQL database, designed for mobile and web applications with real-time sync and offline support.

GC policy (Garbage Collection policy): A Bigtable per-column-family rule that automatically deletes old cell versions, either by keeping only the N most recent versions or deleting data older than N days.

gcloud storage: The modern Cloud Storage CLI surface within the `gcloud` command-line tool, providing commands for managing objects, buckets, lifecycle policies, and IAM bindings.

GENERATION: A unique identifier for a specific version of a versioned Cloud Storage object, used to reference or restore a previous version.

HA (High Availability): A Cloud SQL configuration (`--availability-type=REGIONAL`) that provisions a standby replica in a different zone and performs automatic failover within approximately 60 seconds.

HDD (Hard Disk Drive): A Bigtable storage option with lower cost per GB but higher latency than SSD, suitable for large-volume, throughput-oriented workloads.

HMAC key: A Hash-based Message Authentication Code key used with Cloud Storage for compatibility with tools and libraries that use the S3 API (XML API).

IAM (Identity and Access Management): Google Cloud's system for controlling who (principal) can do what (role/permission) on which resource, enforced through policies at the organization, folder, project, or resource level.

Lifecycle policy: A Cloud Storage configuration containing rules that automatically transition objects to a different storage class or delete them based on conditions such as object age or version count.

Memorystore: Google Cloud's fully managed in-memory data store service, supporting Redis and Memcached, used for caching, session management, and real-time leaderboards.

Nearline (storage class): A Cloud Storage class designed for data accessed at most once per month, with lower storage cost than Standard but a retrieval fee and a 30-day minimum storage duration.

Object versioning: A Cloud Storage bucket feature that retains all previous versions of an object when it is overwritten or deleted, identified by unique generation numbers.

PITR (Point-in-Time Recovery): The ability to restore a Cloud SQL instance to any specific second within the backup retention window, enabled by automated backups plus binary logging (MySQL) or WAL archiving (PostgreSQL).

PostgreSQL: An open-source relational database system; one of the database engines supported by Cloud SQL and the native engine for AlloyDB.

Private IP: An IP address from a private RFC 1918 range, accessible only within a VPC network; recommended for Cloud SQL connections to avoid exposing the database to the public internet.

Processing unit: The unit of compute capacity for Cloud Spanner; 1 node equals 1000 processing units; billed per hour of provisioned capacity.

Pub/Sub: Google Cloud's fully managed, real-time messaging service for asynchronous communication between services using a publish-subscribe pattern.

Read replica: A Cloud SQL instance that replicates data from a primary instance and serves read-only traffic, used to scale read workloads (not for high availability).

Retention policy: A Cloud Storage bucket setting that prevents objects from being deleted or overwritten for a specified minimum duration, used for compliance and data governance.

Signed URL: A time-limited URL that grants access to a specific Cloud Storage object without requiring the requester to have a Google account or IAM permissions; useful for temporary sharing.

SSD (Solid-State Drive): A Bigtable storage option with lower latency than HDD, recommended for latency-sensitive workloads; priced higher per GB.

Spanner: Google Cloud's fully managed, globally distributed relational database service offering strong consistency, horizontal scaling, and 99.999% availability for multi-region configurations.

SQL Server: A relational database engine from Microsoft; one of the database engines supported by Cloud SQL.

Standard (storage class): The default Cloud Storage class with no minimum storage duration and no retrieval fees, best suited for frequently accessed (hot) data.

storage auto-increase: A Cloud SQL feature that automatically increases the storage size of an instance when it is running low on disk space, preventing outages due to full disks.

Uniform bucket-level access: A Cloud Storage bucket setting that disables object-level ACLs and enforces access control exclusively through IAM, simplifying permission management.

VPC (Virtual Private Cloud): Google Cloud's isolated, private virtual network within which compute resources communicate securely; used when configuring Cloud SQL private IP connections.

WAL archiving (Write-Ahead Log archiving): A PostgreSQL mechanism that continuously records all database changes to archive files; required on Cloud SQL PostgreSQL instances to enable point-in-time recovery.